

What you learned:

- A clocked always block describes flip-flops
- Reset behavior becomes visible in waveforms
- Enables are “conditional state updates”

Mini-build 2: PWM dimmer

PWM is a classic because it's simple but feels like “real hardware.”

RTL: pwm.v

```

module pwm #(
    parameter WIDTH = 8
)(
    input          clk,
    input          rst_n,
    input  [WIDTH-1:0] duty,
    output reg     pwm_out
);
    reg [WIDTH-1:0] ctr;

    always @(posedge clk) begin
        if (!rst_n)
            ctr <= {WIDTH{1'b0}};
        else
            ctr <= ctr + 1'b1;
    end

    always @(*) begin
        pwm_out = (ctr < duty);
    end
end

```

```
endmodule
```

Testbench: tb_pwm.v

```

`timescale 1ns/1ps

module tb_pwm;
    reg clk, rst_n;
    reg [7:0] duty;
    wire pwm_out;

    pwm #(.WIDTH(8)) dut (.clk(clk),
        .rst_n(rst_n), .duty(duty), .pwm_out(pwm_out));

    initial clk = 0;
    always #1 clk = ~clk; // faster clock
    for quick waveform patterns

    initial begin
        $dumpfile("waves/pwm.vcd");
        $dumpvars(0, tb_pwm);

        rst_n = 0;
        duty = 8'd0;
        #10;
        rst_n = 1;

        duty = 8'd32; #200;
        duty = 8'd128; #200;
    end
endmodule

```

